Тема 13

- шаблони/templates – compile-time параметричен полиморфизъм
- параметричен полиморфизъм – функции/класове с общо предназначение спрямо типовете

```
template <typename class T>
const T& max(const T& lhs, const T& rhr)
        return lhs<rhs ? rhs : lhs;
```

→ при извикване на max подаваме типа като параметър → max<int>(4, 5) → създава се копие на шаблона, в което T е заместено с конкретния тип

- необходима функция на тип – функция, която типът трябва да притежава, за може кодът да се генерира

→ при max<T> необходима функция е оп. <

- проблем при разделна компилация – .cpp файловете виждат само шаблонния код в .h, но не виждат различните имплементации

→ решения:

1) в края на .cpp файла на шаблонния клас/ф-я разписваме с какви типове ще ползваме кода

2) използване .hpp файл без разделна компилация

```
A.h
template <class T>
class A{...};
A.cpp
:
template A<int>
template A<char>
```

• шаблонна специализация - клас/ф-я с различно поведение за конкретен тип

пример: vector<bool> използва битсет

<temp T> sort(T* arr, size) // quick+insertion sort → temp<> sort(char* arr, size) // count sort

→ примери за шаблони от стандартната библиотека: vector, queue, sort, find, smart pointers

---

• умни указатели - обвиващи класове на указателите, които мениджират паметта на обектите, към които сочат

→ unique_ptr - точно 1 указател за точно 1 обект

⇒ К.К. и ОП= са забранени

М.К. и М.ОП= са разрешени

unique_ptr<A> ptrA = make_unique<A>(3,4,5);

↳ wrapper за new

→ честяване new и delete

→ shared_ptr - 1 обект, много указатели

→ паметта се заделя при създаване на първия указател и се освобождава при изтриване на последния

→ в класа има и указател към int в дин. памет, който пази броя на живите указатели

→ в деструктора int* и T* се трият, само ако броят на живите указатели е 0 (или 1, ако фактът се намаля след проверката)

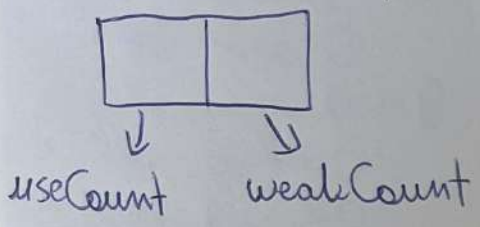
→ weak_ptr (non-owning reference) - указател към обект, който е мениджиран от shared_ptr

не влияят на триенето

→ може да са насочени към обект, който вече е изтрит
⇒ трябва да има проверка дали обектът още е жив

→ възможно е промотиране в shared_ptr

→ имплементиране чрез наредена двойка числа



useCount - брой shared_ptr
weakCount - брой weak_ptr | +0, ако use=0
| +1, ако use>0

Броятъм се трие, когато weakCount=0
Обектът се трие, когато useCount=0

• Пример за stack с фиксиран размер

```
#define st size_t
~~class Stack~~ template <class T, const unsigned N>
class Stack {
    T data[N];
    st size=0;
public:
    void push(const T&);
    void push(T &&);
    const T& peek() const;
    void pop();
    bool isEmpty() const;
    bool isFull() const;
};
```